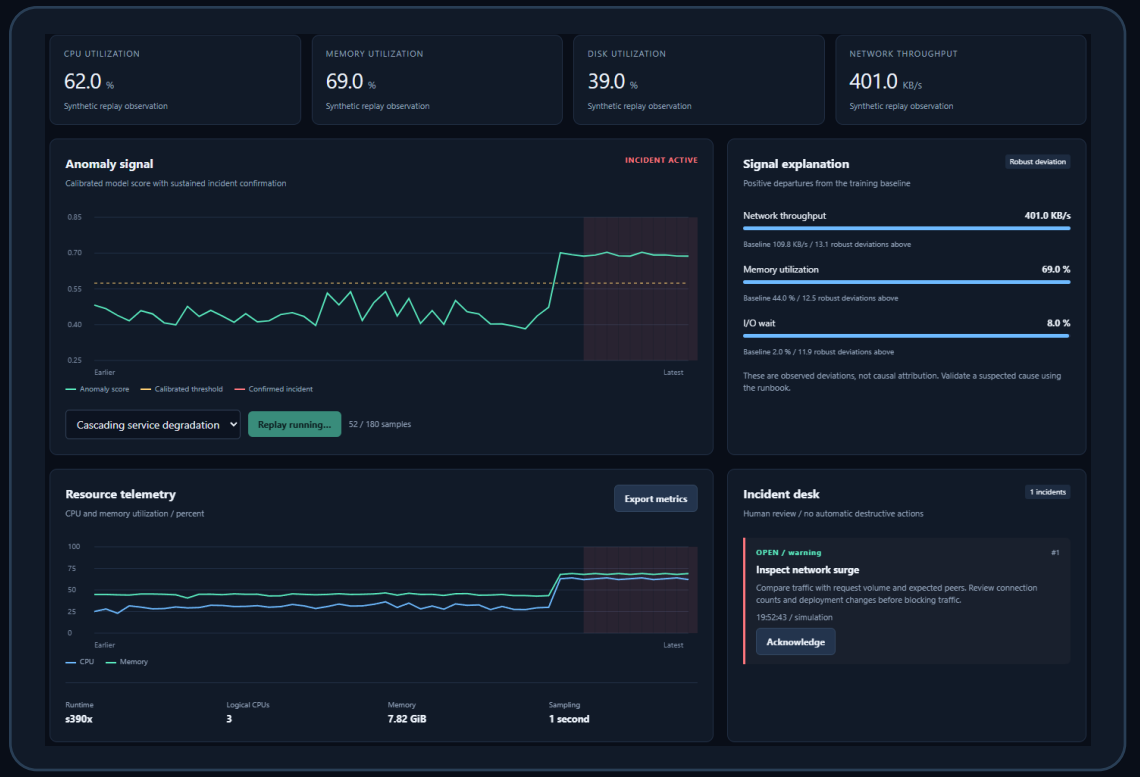
LINUXONE OPERATIONS INTELLIGENCE

See resource stress. Understand the signal.

Local AI monitoring that turns sustained resource changes into explainable incidents for small operations teams.

Real telemetry. Local inference.

A working s390x deployment on IBM LinuxONE.



Team V_IBM Z Datathon 2026 Project

THE PROBLEM AND BENEFICIARIES

Small teams need a clear path from signal to action.

01

Fragmented signals

CPU, memory, network, and I/O measurements need context across time and across signals.

02

Alert overload

Brief spikes can distract operators. A sustained incident should carry evidence and a review runbook.

03

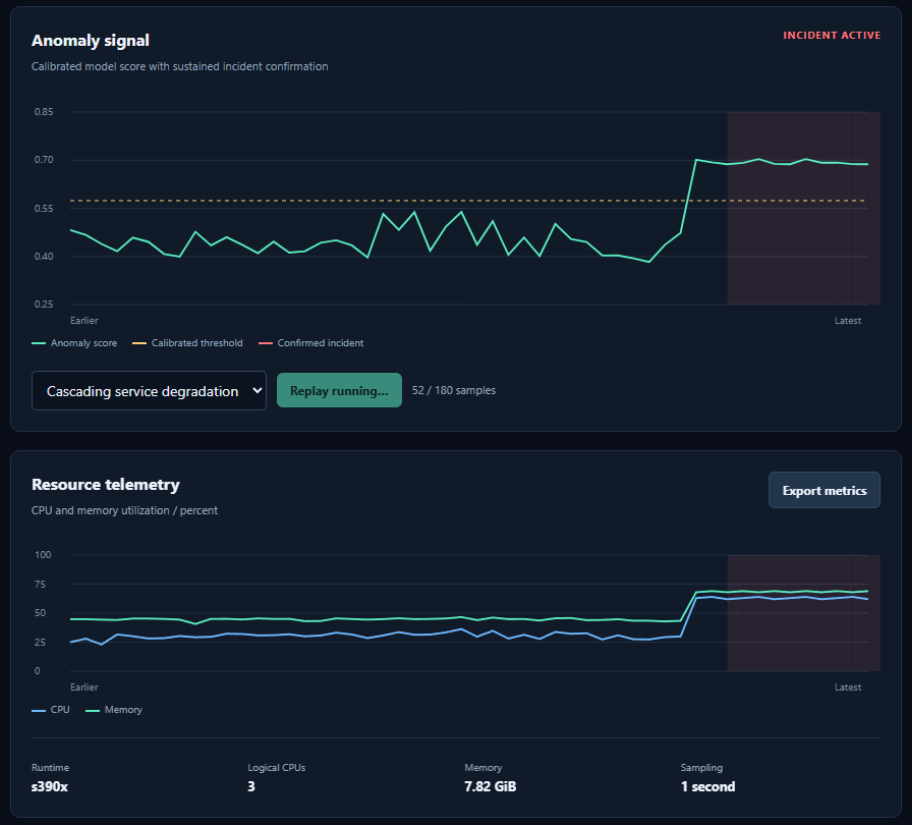
Limited capacity

Campus, community, and small business services often rely on small technical teams with limited on-call time.

Expected value: faster triage and clearer operational decisions. Outage reduction and business ROI require validation with real service operators.

WORKING PROTOTYPE

Observe. Score. Confirm. Explain.



Observe six resource signals

Live VM counters are sampled approximately once per second.

Confirm a sustained incident

Three abnormal samples open an incident; three clear samples resolve it.

Make review actionable

Baseline deviations, a runbook, acknowledgment, and exports support human decisions.

Screen shown: controlled synthetic replay. The live monitor has a separate model and real VM measurements.

MODEL AND REPRODUCIBLE EVIDENCE

The model catches a coordinated shift below limits.

10/10

Synthetic events detected

9/10

Static guardrail comparator

6.89 ms

Maximum scenario inference p95

The first compound episode raises several metrics together while staying below critical guardrails. Isolation Forest detects the departure; static rules alone miss this one event.

Evaluation uses 800 synthetic training samples, 256 independent calibration samples, and six held-out 180-sample scenarios. Normal control: zero confirmed incidents. Stress episodes: median confirmation delay of two sample steps.

These are controlled synthetic results, not production accuracy or a service SLA. Full counts, seeds, and runtime are published in evidence/evaluation.json.

IBM Z / LINUXONE ARCHITECTURE

Keep telemetry and inference on LinuxONE.

Collect
psutil / 6 metrics

>

Infer
128-tree Isolation Forest

>

Persist
SQLite WAL / audit

>

Review
FastAPI / browser

Deployed and measured on s390x, Python 3.12.15, 3 logical CPUs, and 7.82 GiB VM memory. The app container has a 1.5-CPU / 2-GiB limit and runs with a read-only source mount and reduced privileges.

Live training: 80 real observations plus 40 calibration observations. Replay training is isolated. Raw telemetry stays local and no external AI inference API is required.

CPU inference only. No claim of Telum acceleration, protected execution, or high availability.

ENGINEERING AND PRACTICAL LIMITS

Reviewable evidence from a working deployment.

12 checks passed

Authentication, foreign-origin mutation rejection, invalid metrics, healthy control, spike suppression, incident recovery, and model isolation.

Operational controls

Token-protected API, versioned baseline metadata, acknowledgment, persisted incidents and audit records, bounded sample retention, restart recovery.

Explicit limitations

Short live baseline, synthetic stress evaluation, single-VM architecture, and a temporary HTTP deployment. Validate longer baselines and TLS/SSO before operational use.

No destructive remediation is automated. Operators validate the suspected cause before taking action.

Explanations are robust baseline deviations, not causal attribution.

WHAT COMES NEXT

A practical foundation for essential-service operations.

Today: live LinuxONE collector, local ML, incident lifecycle, failure replay, reproducible evaluation, authenticated dashboard, and exportable evidence.

Next: representative long-term baselines; signed multi-host agents; TLS + organizational SSO/RBAC; drift review; alert routing; and availability testing with service owners.

Source, evidence, architecture, and setup

https://github.com/Hemanthk101/Team-V-ZWatch-LinuxONE

AI assisted implementation and documentation. Team V reviews and owns the final submission.